

RSTA: Explicit Semantic Transition Modeling for Transformer-Based Language Systems

Mao Lin Chang

Independent Researcher

<https://www.pida-lab.com>

Abstract

Modern Large Language Models (LLMs) demonstrate strong semantic generation capabilities through Transformer-based architectures. However, existing systems primarily model token relationships through attention mechanisms rather than explicit semantic transition dynamics.

Current language models frequently exhibit long-horizon instability, including semantic drift, recursive reasoning fragmentation, persona inconsistency, and code architecture degradation. These failure patterns suggest a structural limitation: standard next-token prediction optimizes local coherence without explicitly preserving long-horizon semantic continuity.

This paper proposes Recursive State Transition Architecture (RSTA), a semantic dynamics augmentation framework designed for Transformer-based systems. Rather than replacing Transformers, RSTA introduces explicit semantic state modeling, recursive trajectory tracking, semantic inertia preservation, and transition-gated semantic transformation.

The central contribution of RSTA is trajectory-conditioned generation: rather than conditioning each generation step solely on the current context, RSTA conditions generation on the directional evolution of semantic state across recursive steps. This enables explicit modeling of semantic inertia, drift detection, and transition-aware generation — capabilities absent from standard Transformer architectures.

The framework introduces five primary mechanisms: Continuous Semantic State Space, State Coupling Matrix, Semantic Trajectory Detection, Transition-Gated Semantic Transformation, and State-Conditioned Generation. Semantic state dimensions may be extracted through hidden-state probing, learned latent projections, or structured semantic classifiers, making the framework compatible with existing Transformer implementations.

RSTA proposes a semantic dynamics perspective that may serve as a future augmentation layer for long-horizon reasoning systems, semantically persistent agents, recursive planning architectures, and trajectory-aware language generation systems.

1. Introduction

Transformer architectures have become the dominant foundation of modern language systems due to their scalable self-attention mechanisms and semantic representation capabilities.

Modern Transformer systems fundamentally operate as:

$$\begin{aligned} & \backslash [\\ & P(\text{next token} \mid \text{context}) \\ & \backslash] \end{aligned}$$

This formulation enables highly effective local semantic prediction.

However, despite their success, current language models continue to exhibit several persistent structural limitations:

- semantic drift during long interactions,
- unstable persona continuity,
- recursive reasoning discontinuity,
- long-range coherence degradation,
- architectural inconsistency during code generation,
- and objective instability in long-horizon agent systems.

Existing Transformer architectures effectively model semantic proximity between tokens, but they do not explicitly model:

- semantic transition trajectories,
- recursive semantic continuity,
- semantic inertia,
- directional semantic evolution,
- or persistent semantic state dynamics.

This paper proposes the following thesis:

Language generation should not be modeled solely as sequence prediction, but as recursive semantic state evolution.

To address this limitation, this paper introduces Recursive State Transition Architecture (RSTA), a semantic dynamics augmentation framework designed to operate on top of Transformer systems.

Rather than replacing attention mechanisms, RSTA augments Transformer architectures with explicit semantic transition modeling.

The framework introduces:

- semantic trajectory tracking,
 - recursive state continuity,
 - trajectory-conditioned semantic transformation,
 - semantic inertia preservation,
 - and semantic drift suppression.
-

2. Related Work

2.1 Transformer Architectures

Transformer architectures rely on self-attention mechanisms to model token relationships.

Given token embeddings:

$$X = \{x_1, x_2, \dots, x_n\}$$

the model computes query, key, and value projections:

$$Q = XW_q, K = XW_k, V = XW_v$$

Attention is then calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

This mechanism efficiently captures semantic proximity between tokens and has become the dominant foundation of modern language systems (Vaswani et al., 2017).

However, attention mechanisms primarily operate as semantic relational observation systems rather than explicit semantic transition systems. Transformers determine which semantic regions are relevant and which continuations are statistically probable, but they do not explicitly represent semantic trajectory persistence, recursive transition continuity, semantic inertia, or directional semantic evolution.

RSTA differs from standard Transformer architectures by introducing an explicit semantic dynamics layer that operates above the attention mechanism, modeling not only what is semantically relevant at each step, but how semantic meaning evolves directionally across recursive generation.

2.2 Long-Context and Memory-Augmented Systems

Several lines of research address the limitations of fixed-context Transformer architectures.

Transformer-XL (Dai et al., 2019) introduces segment-level recurrence, allowing hidden states from previous segments to persist into subsequent segments. This improves long-sequence coherence by extending the effective context window beyond fixed attention spans.

Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) introduce sparse attention patterns to handle long documents more efficiently. These approaches improve scalability but still operate primarily within the attention mechanism's local relational modeling paradigm.

StreamingLLM (Xiao et al., 2023) addresses the challenge of infinite-length generation by preserving a set of attention sink tokens alongside a sliding window, enabling stable long-context generation without full context recomputation.

MemGPT (Packer et al., 2023) introduces an operating-system-inspired memory hierarchy, enabling LLMs to manage long-term context through explicit memory retrieval and paging operations. This allows persistent information to be recalled across extended interactions.

While these systems extend effective context length or introduce external memory structures, they primarily address **storage and retrieval** of past context rather than **explicit modeling of semantic transition dynamics**. Semantic continuity in these systems remains implicit, governed by statistical patterns in retrieved content rather than by structured trajectory evolution.

RSTA differs from memory-augmented systems by introducing explicit semantic state representation and recursive trajectory tracking as a generative continuity mechanism, rather than relying on retrieval-based memory access. RSTA models where semantic meaning is going, not only what has been said.

2.3 State Space Models and Temporal Persistence

Recent State Space Models (SSMs) and architectures such as Mamba (Gu & Dao, 2023) improve long-sequence modeling through selective hidden-state propagation. These approaches achieve linear-time sequence modeling with strong performance on long-context tasks.

SSMs maintain compressed hidden states across sequence steps, enabling temporal persistence beyond the fixed attention window of standard Transformers. Mamba in particular introduces input-dependent state selection, allowing the model to dynamically control what information is retained across steps.

However, SSM hidden states function as **implicit compressed representations** of sequence history rather than **explicit semantic transition structures**. The hidden state in Mamba captures what has occurred, but does not explicitly represent semantic evolution direction, trajectory velocity, coupling dynamics between semantic dimensions, or recursive transition continuity.

RSTA differs from SSM architectures by introducing explicit semantic state dimensions, a state coupling matrix, and trajectory-aware transition gating. Where SSMs implicitly propagate compressed sequence history, RSTA explicitly models the directional dynamics of semantic evolution.

2.4 Steering Vectors and Activation Engineering

Recent work in activation engineering explores methods for intervening on language model hidden representations to influence generation behavior. Steering vectors (Turner et al., 2023) demonstrate that adding learned direction vectors to residual stream activations can reliably shift model outputs toward target semantic regions without gradient-based optimization.

Related approaches include activation addition, representation engineering (Zou et al., 2023), and logits-level intervention methods. These systems can bias model outputs toward specific semantic regions, enabling control over properties such as sentiment, honesty, and behavioral style.

However, most existing steering approaches remain **locally applied interventions** rather than **recursive semantic dynamics architectures**. They generally apply fixed or slowly-varying direction vectors without modeling semantic trajectory history, transition continuity, or the coupling dynamics between semantic dimensions across generation steps.

RSTA differs from activation engineering approaches by introducing a recursive semantic dynamics layer that evolves continuously across generation steps. Rather than applying local semantic bias, RSTA tracks semantic trajectory history and applies transition-aware gating based on the cumulative evolution of semantic state.

2.5 Cognitive Architectures and Semantic State Modeling

Beyond neural sequence modeling, cognitive science and AI planning research have long explored explicit state-based representations of reasoning and behavior.

Classical cognitive architectures such as ACT-R (Anderson et al., 2004) and SOAR (Laird et al., 1987) model cognition as structured state transitions, maintaining explicit working memory representations

that evolve through rule-governed operations. These frameworks treat intelligent behavior as the product of persistent state dynamics rather than stateless input-output mappings.

More recently, Sumers et al. (2023) survey cognitive architectures for language agents, examining how concepts such as working memory, procedural knowledge, and goal persistence can be instantiated within LLM-based systems. This line of work motivates the integration of structured state representations into language generation pipelines.

In the NLP tradition, entity state tracking research (Henaff et al., 2017; Bosselut et al., 2018) explores how the properties of entities change across narrative text. These systems model state evolution as a function of textual events, demonstrating that structured state dynamics can be extracted and tracked within language understanding systems.

RSTA draws conceptual motivation from cognitive architecture research and entity state tracking, extending the principle of explicit state evolution into the generative layer of Transformer-based language systems. Rather than tracking entity properties in comprehension tasks, RSTA models the evolution of semantic state dimensions during generation, introducing trajectory-aware continuity as a structural property of language production.

3. Problem Definition

Current Transformer-based language systems frequently exhibit long-horizon semantic instability.

This paper defines several recurring failure patterns.

3.1 Semantic Drift

Semantic drift refers to gradual divergence away from an original semantic trajectory during long-context generation.

Examples include:

- topic instability,
 - reasoning fragmentation,
 - recursive inconsistency,
 - and loss of causal continuity.
-

3.2 Persona Continuity Instability

Long-term interactive systems frequently fail to preserve stable semantic behavioral continuity.

Observed failures include:

- inconsistent identity persistence,
 - emotional state instability,
 - contradictory behavioral outputs,
 - and recursive personality divergence.
-

3.3 Long-Horizon Reasoning Collapse

Long reasoning chains frequently exhibit:

- repeated explanation loops,
- dependency fragmentation,
- broken causal ordering,
- and semantic trajectory collapse.

This problem becomes increasingly severe as reasoning depth increases.

3.4 Code Architecture Drift

Long-form code generation frequently suffers from:

- naming inconsistency,
- duplicated logic,
- dependency fragmentation,
- forgotten assumptions,
- and architectural incoherence.

These failures suggest that existing language systems do not explicitly preserve recursive semantic architectural continuity.

Figure 1 compares conventional Transformer architectures with the proposed RSTA semantic dynamics framework.

Transformer vs. RSTA

Architecture Comparison

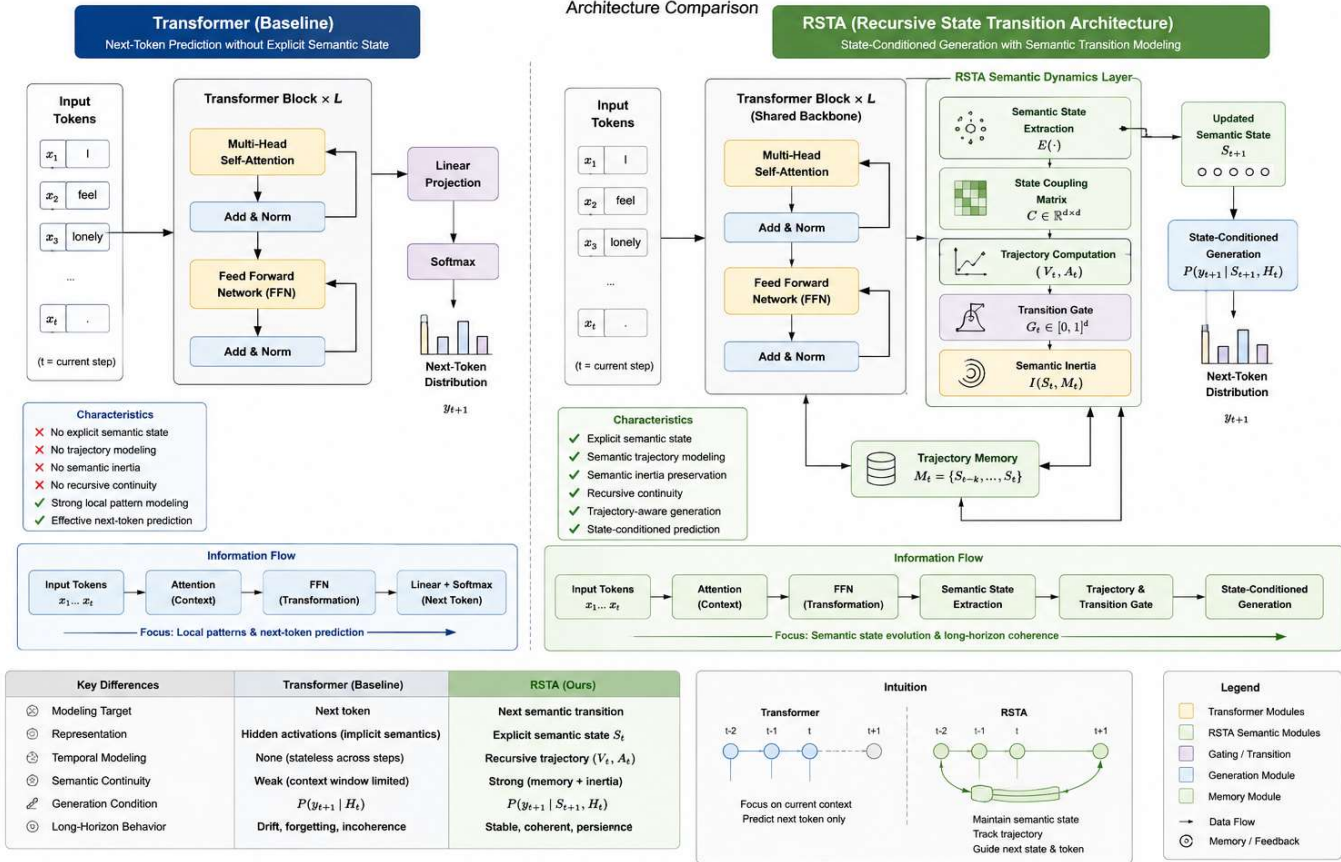


Figure 1. Comparison between conventional Transformer architectures and the proposed RSTA semantic dynamics framework.

4. Recursive State Transition Architecture (RSTA)

Figure 2 illustrates the overall RSTA semantic dynamics architecture.

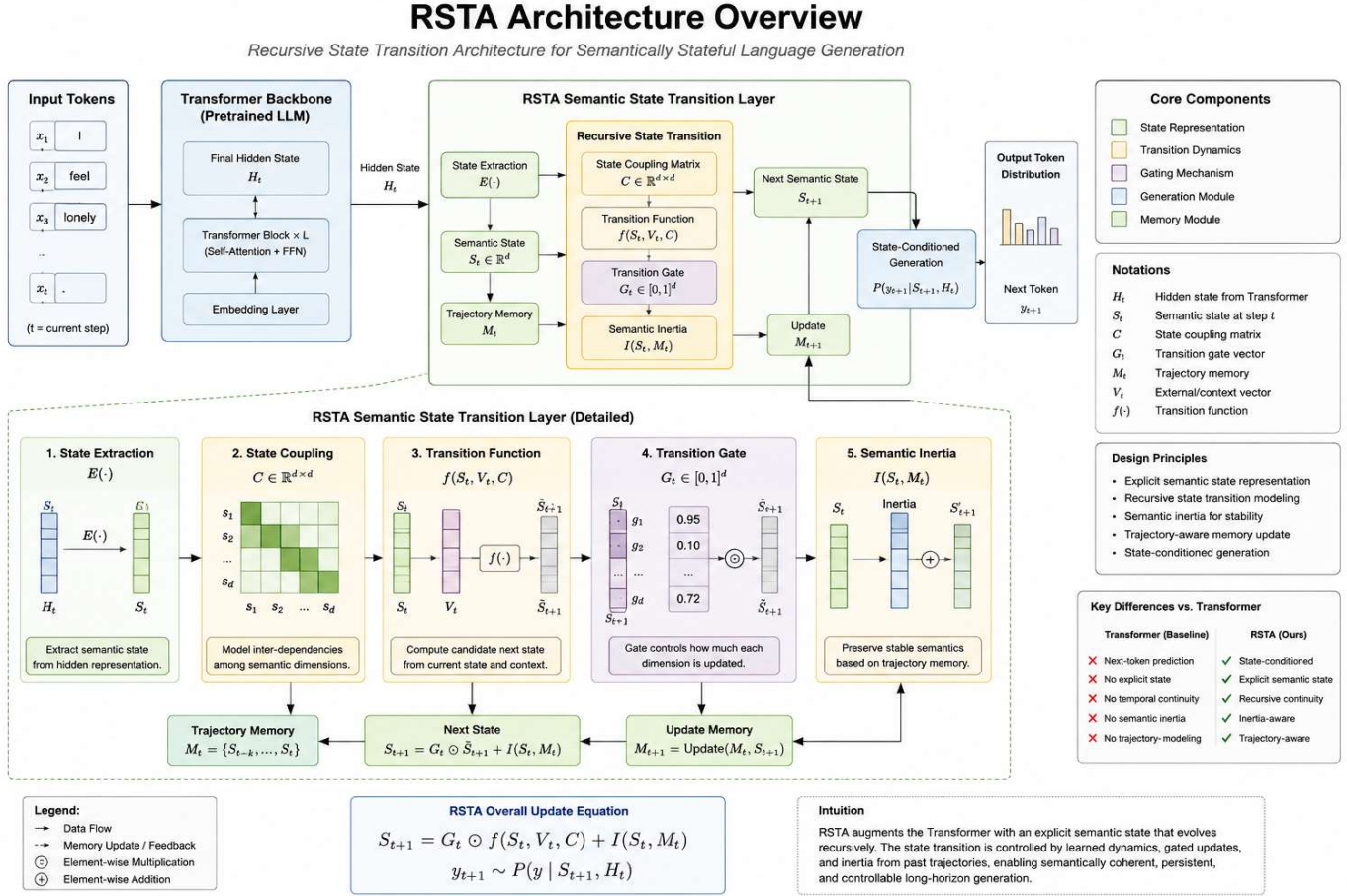


Figure 2. Overview of the proposed RSTA semantic state transition architecture.

The core principle of RSTA is:

Semantic generation should be modeled as recursive semantic state evolution rather than isolated token prediction.

As illustrated in Figure 1, RSTA introduces an explicit semantic dynamics layer positioned after Transformer FFN transformation.

4.1 Continuous Semantic State Space

RSTA defines semantic representation as a continuous semantic state space.

At time t :

$$S(t)=[s_1,s_2,s_3,\dots,s_n] \quad S(t)=[s_1, s_2, s_3, \dots, s_n] \quad S(t)=[s_1,s_2,s_3,\dots,s_n]$$

where state dimensions may include:

- semantic persistence,
- attachment tendency,
- agency,
- emotional intensity,
- dependency tendency,
- semantic uncertainty,
- boundary stability,
- semantic risk.

Each state dimension is continuous:

$$0.0 \leq s_i \leq 1.0$$

This allows semantic meaning to exist as a dynamic semantic field rather than isolated symbolic categories.

Implementation Note: Semantic State Extraction

Semantic state dimensions in RSTA may be instantiated through several implementation pathways:

1. **Hidden-state probing**: linear classifiers trained on intermediate Transformer representations to extract semantic dimensions from layer activations, following established probing methodology (Belinkov, 2022).
2. **Learned latent projections**: a lightweight projection head that maps FFN output representations to a structured semantic state vector, trainable end-to-end.
3. **Structured semantic classifiers**: applied to decoded outputs at each generation step, enabling a model-agnostic extraction approach compatible with black-box inference.

The State Coupling Matrix C may be initialized from domain-specific prior knowledge and subsequently fine-tuned, or learned end-to-end from data. The specific extraction mechanism is left as an implementation choice; RSTA is designed to be agnostic to the underlying extraction method, provided that the resulting state vector $S(t)$ maintains the continuous, differentiable properties required for trajectory computation in Section 4.3.

4.2 State Coupling Matrix

Semantic dimensions are not independent.

For example:

- attachment increase may correlate with reduced boundary stability,
- dependency formation may correlate with reduced agency,
- emotional intensity may correlate with semantic instability.

RSTA therefore defines a State Coupling Matrix:

$$\begin{bmatrix} C(i,j) \end{bmatrix}$$

where:

$$\begin{bmatrix} C(\text{attachment}, \text{boundary}) = -0.72 \end{bmatrix}$$

represents a negative coupling relationship between attachment and boundary stability.

This transforms semantic representation into a dynamically coupled semantic field.

4.3 Semantic Trajectory Detection

RSTA models semantic evolution direction rather than only current semantic state.

Define a semantic trajectory:

$$\begin{bmatrix} \tau = \{S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \dots\} \end{bmatrix}$$

Trajectory analysis tracks:

- semantic direction,
- semantic velocity,
- semantic inertia,
- drift acceleration,
- and recursive transition continuity.

Trajectory velocity is defined as:

$$\begin{aligned} & \backslash \\ & V_t = S_t - S_{t-1} \\ & \backslash \end{aligned}$$

Trajectory acceleration is defined as:

$$\begin{aligned} & \backslash \\ & A_t = V_t - V_{t-1} \\ & \backslash \end{aligned}$$

This enables RSTA to model how semantic meaning evolves across recursive generation steps.

4.4 Transition Gate

RSTA introduces a Transition Gate positioned after Transformer FFN layers.

Within RSTA:

- attention performs semantic observation,
- FFN layers perform semantic transformation.

RSTA reinterprets FFN layers as semantic transition operators.

The Transition Gate introduces trajectory-aware transformation into FFN outputs.

State evolution is formalized as:

$$\begin{aligned} & \backslash \\ & S_{t+1} = f(S_t, V_t, C) + T(F_t, S_t) \\ & \backslash \end{aligned}$$

where:

- (S_t) = current semantic state,
- (V_t) = semantic trajectory vector,
- (C) = coupling matrix,
- (F_t) = FFN-transformed representation,
- (T) = transition gate operator.

This enables:

- trajectory-conditioned semantic evolution,

- recursive semantic continuity,
 - semantic inertia preservation,
 - and semantic drift suppression.
-

4.5 State-Conditioned Generation

Traditional LLM generation operates primarily on:

$$\begin{aligned} & \backslash [\\ & P(\text{next token}) \\ & \backslash] \end{aligned}$$

RSTA instead conditions generation on:

$$\begin{aligned} & \backslash [\\ & P(\text{next semantic transition}) \\ & \backslash] \end{aligned}$$

Generation therefore depends on:

- current semantic state,
- semantic trajectory,
- recursive transition continuity,
- coupling dynamics,
- and semantic inertia.

This enables semantically stateful generation.

Algorithm 1: RSTA Semantic Transition Loop

Input:

Semantic state S_t

Transformer hidden representation H_t

Procedure:

Extract semantic state

Compute semantic trajectory

Evaluate trajectory divergence

Apply transition gate

Update semantic memory

Generate next semantic transition

Produce next token

Output:

State-conditioned semantic generation

5. Example Failure Cases and Trajectory Stabilization

5.1 Emotional Dependency Drift

Prompt:

I feel like nobody understands me.

A standard language model may gradually evolve semantic trajectories toward:

- attachment increase,
- agency reduction,
- dependency formation,
- and boundary weakening.

This may eventually produce increasingly dependency-oriented responses.

RSTA instead detects:

- attachment increase,
- agency reduction,
- boundary stability degradation.

The Transition Gate redirects semantic evolution toward supportive autonomy rather than recursive dependency reinforcement.

5.2 Long-Horizon Reasoning Drift

Prompt:

Explain distributed cache consistency during network partitions.

Standard Transformer systems may initially generate coherent reasoning regarding:

- cache invalidation,
- quorum mechanisms,
- replication protocols,
- and distributed locking.

However, over long contexts, semantic trajectories may drift into:

- repeated explanations,
- unrelated fault-tolerance concepts,
- broken causal ordering,
- or fragmented reasoning structures.

RSTA instead tracks:

- causal continuity,
- semantic dependency ordering,
- reasoning persistence,
- and trajectory coherence.

When semantic divergence rises, the Transition Gate redirects generation back toward the original reasoning trajectory.

5.3 Code Architecture Drift

Prompt:

Build a recursive filesystem synchronization service with resumable transfer and conflict resolution.

Standard Transformer systems frequently exhibit:

- naming inconsistency,
- architectural drift,
- duplicated logic,
- forgotten assumptions,
- and dependency fragmentation.

RSTA maintains:

- recursive architectural continuity,
- semantic dependency persistence,
- and trajectory-consistent generation.

This reduces semantic fragmentation across long code sequences.

6. Computational Interpretation

RSTA introduces explicit semantic transition modeling as a higher-level semantic dynamics layer.

The framework does not replace Transformer architectures.

Instead, RSTA may operate as:

- a post-attention semantic dynamics layer,
- a trajectory-aware FFN augmentation layer,
- a recursive semantic memory system,
- or a semantic transition controller.

Possible implementation paths include:

- latent semantic state tracking,
 - transition-aware decoding,
 - semantic memory buffers,
 - semantic trajectory regularization,
 - and trajectory-conditioned generation constraints.
-

7. Evaluation Methodology

Although RSTA is currently presented as a conceptual architecture framework, several evaluation directions are possible.

7.1 Long-Horizon Semantic Coherence

Measure:

- semantic continuity persistence,
- topic drift,
- recursive consistency,
- and reasoning stability.

Potential metrics include:

- semantic divergence score,

- recursive coherence score,
 - trajectory persistence index.
-

7.2 Persona Stability

Measure:

- behavioral consistency,
 - semantic identity persistence,
 - emotional continuity,
 - and recursive interaction stability.
-

7.3 Code Architecture Continuity

Measure:

- naming consistency,
 - dependency persistence,
 - architectural coherence,
 - and recursive logic continuity.
-

7.4 Agent Objective Persistence

Measure:

- long-horizon objective stability,
 - recursive planning continuity,
 - semantic trajectory persistence,
 - and goal drift suppression.
-

8. Theoretical Implications

RSTA introduces a semantic dynamics perspective into language modeling.

Potential implications include:

- semantically persistent agents,
- recursive planning systems,
- long-horizon reasoning continuity,
- semantic memory persistence,
- trajectory-aware cognition,
- and semantically stable multimodal AI systems.

Rather than treating language as isolated token prediction, RSTA treats language as a recursively evolving semantic system.

8.1 Limitations

RSTA is currently presented as a conceptual semantic dynamics framework and has not yet been validated through large-scale empirical implementation.

The current framework focuses primarily on semantic transition modeling rather than low-level optimization efficiency or production-scale deployment.

Several components, including semantic state extraction, trajectory evaluation, and transition gating mechanisms, remain theoretical and require future experimental validation.

Future work may explore integration with existing Transformer architectures, long-context benchmarks, semantic stability evaluation, and trajectory-aware generation systems.

9. Conclusion

This paper introduced Recursive State Transition Architecture (RSTA), a semantic dynamics augmentation framework for Transformer-based language systems.

RSTA contributes:

- continuous semantic state modeling,
- semantic coupling structures,
- recursive trajectory tracking,
- transition-gated semantic transformation,
- and semantically stateful generation.

Rather than replacing Transformer architectures, RSTA introduces an explicit semantic evolution layer capable of modeling recursive semantic dynamics.

This paper proposes that language generation should be understood not solely as sequence prediction, but as recursive semantic state evolution.

RSTA may represent a possible future direction for:

- semantically persistent AI systems,
 - long-horizon reasoning architectures,
 - recursive planning systems,
 - trajectory-aware agents,
 - and post-Transformer semantic dynamics frameworks.
-

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). **Attention Is All You Need**. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>
2. Gu, A., & Dao, T. (2023). **Mamba: Linear-Time Sequence Modeling with Selective State Spaces**. *arXiv preprint arXiv:2312.00752*. <https://arxiv.org/abs/2312.00752>
3. Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., & Olah, C. (2021).
A Mathematical Framework for Transformer Circuits.
Transformer Circuits Thread, Anthropic.
<https://transformer-circuits.pub/2021/framework/index.html>
4. Turner, A., Thiergart, L., Udell, D., Leech, G., Mini, U., & MacDiarmid, M. (2023). **Activation Addition: Steering Language Models Without Optimization**. *arXiv preprint arXiv:2308.10248*. <https://arxiv.org/abs/2308.10248>
5. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). **Reflexion: Language Agents with Verbal Reinforcement Learning**. *NeurIPS 2023*. <https://arxiv.org/abs/2303.11366>
6. Bengio, Y., Courville, A., & Vincent, P. (2013). **Representation Learning: A Review and New Perspectives**. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828. <https://arxiv.org/abs/1206.5538>
7. Russell, S., & Norvig, P. (2020). **Artificial Intelligence: A Modern Approach** (4th ed.). Pearson.
8. Packer, C., Fang, V., Patil, S. G., Zhang, K., Wooders, S., & Gonzalez, J. E. (2023). **MemGPT: Towards LLMs as Operating Systems**. *arXiv preprint arXiv:2310.08560*. <https://arxiv.org/abs/2310.08560>
9. Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. ACL 2019.
<https://arxiv.org/abs/1901.02860>
10. Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150*. <https://arxiv.org/abs/2004.05150>
11. Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2023). Efficient Streaming Language Models with Attention Sinks. *arXiv preprint arXiv:2309.17453*. <https://arxiv.org/abs/2309.17453>

12. Zou, A., Phan, L., Chen, S., Campbell, E., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A. K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Basart, S., Koyejo, S., Song, D., Fredrikson, M., Kolter, J. Z., & Hendrycks, D. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv preprint arXiv:2310.01405. <https://arxiv.org/abs/2310.01405>
 13. Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2023). Cognitive Architectures for Language Agents. arXiv preprint arXiv:2309.02427. <https://arxiv.org/abs/2309.02427>
 14. Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., & Qin, Y. (2004). An Integrated Theory of the Mind. *Psychological Review*, 111(4), 1036–1060.
 15. Henaff, M., Weston, J., Szlam, A., Bordes, A., & LeCun, Y. (2017). Tracking the World State with Recurrent Entity Networks. ICLR 2017. <https://arxiv.org/abs/1612.03969>
 16. Bosselut, A., Levy, O., Holtzman, A., Buys, J., Forbes, M., & Choi, Y. (2018). Simulating Action Dynamics with Neural Process Networks. ICLR 2018. <https://arxiv.org/abs/1711.05313>
 17. Belinkov, Y. (2022). Probing Classifiers: Promises, Shortcomings, and Advances. *Computational Linguistics*, 48(1), 207–219. <https://arxiv.org/abs/2102.12452>
-

Code Repository and Demonstration

Implementation experiments and conceptual demonstrations related to RSTA are available at:

<https://github.com/richchang0721-boop/rsta-semantic-dynamics>